PES-VCS Lab Report

Sai Harshith <PES1UG24CS398>

Repository implementation for the OS Unit 4 Orange Problem: content-addressable object storage, recursive tree objects, staging index, commits, history, references, and analysis questions.

Phase	Implemented
1	object_write, object_read, hashing, sharding, atomic writes, integrity verification
2	tree serialization usage and recursive tree_from_index
3	index_load, index_save, index_add with metadata and atomic writes
4	commit_create with tree snapshots, parent links, author metadata, and HEAD update

Required Screenshots

Screenshot 1A: ./test_objects output showing all Phase 1 tests passing

```
Screenshot 1A
./test_objects output showing all Phase 1 tests passing

Stored blob with hash: d58213f5dbe0629b5c2fa28e
5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12

Object stored at: .pes/objects/d5/8213f5dbe0629b
5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12


PASS: blob storage

PASS: deduplication

PASS: integrity check



All Phase 1 tests passed.
```

Screenshot 1B: find .pes/objects -type f showing sharded object storage

```
Screenshot 1B
find .pes/objects -type f showing sharded object storage

.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7a
e34b337afedb9b46a1af8e11ec4f476

.pes/objects/2a/594d39232787fba8eb7287418aec99c8
fc2ecdaf5aaf2e650eda471e566fcf

.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea
09227ed0221bbe9db5e5c4b9aafc12
```

Screenshot 2A: ./test_tree output showing all Phase 2 tests passing

```
Screenshot 2A
./test_tree output showing all Phase 2 tests passing

Serialized tree: 139 bytes

PASS: tree serialize/parse roundtrip

PASS: tree deterministic serialization



All Phase 2 tests passed.
```

Screenshot 2B: xxd of a raw tree object showing the binary tree format

```
Screenshot 2B
xxd of a raw tree object showing the binary tree format

00000000:  7472  6565  2034  3600  3130  3036  3434
206d    tree  46.100644  m

00000010:  6169  6e2e  6300  fd8a  0f23  eb49  4c98
e85b    ain.c....#.IL..[

00000020:  3819  a691  d2aa  6a5c  c7ff  2d65  263f
cf93    8.....j\...-e&?..

00000030:  f35c  8639  d7c7
      .\.9..
```

Screenshot 3A: pes init, pes add, pes status sequence

```
Screenshot 3A
pes init, pes add, pes status sequence

$ ./pes init
Initialized empty PES repository in .pes/
$ ./pes add file1.txt file2.txt
$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)
```

Screenshot 3B: cat .pes/index showing the text-format index

```
Screenshot 3B
cat .pes/index showing the text-format index

$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776691341 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776691341 6 file2.txt
```

Screenshot 4A: pes log output showing three commits

Screenshot 4A
pes log output showing three commits

```
$ ./pes log
commit 64b1346763a3bf82cab91172264eb7f8c99b072d0c0f01b9f04f105c9741fc95
Author: Sai Harshith <PES1UG24CS398>
Date:   1776691361

    Add farewell

commit c66b8bf05f6620222172e4c0902d3d5fe601515d3b88f1717f09739729489792
Author: Sai Harshith <PES1UG24CS398>
Date:   1776691361

    Add world

commit 0e177f02433419afb7ba6275fdcd5b68dd8a7629a77b12850663da7414a3e1b3
Author: Sai Harshith <PES1UG24CS398>
Date:   1776691361

    Initial commit
```

Screenshot 4B: find .pes -type f | sort after three commits

Screenshot 4B
find .pes -type f | sort after three commits

```
$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/0e/177f02433419afb7ba6275fdcd5b68dd8a7629a77b12850663da7414a3e1b3
.pes/objects/4f/3441f3d4f853d6f7abc24edceb1e2bb7d267458f5ffe42d8eed7511018444d
.pes/objects/5b/4a827e55ead9d987ae1d141f6473d3e0be7136156059f4fbcd867974505736
.pes/objects/64/b1346763a3bf82cab91172264eb7f8c99b072d0c0f01b9f04f105c9741fc95
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/c6/6b8bf05f6620222172e4c0902d3d5fe601515d3b88f1717f09739729489792
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ff/69f1fffcac805e87dae9e74b9c0baacb773f267826947d7c957e23b4aef923
.pes/refs/heads/main
```

Screenshot 4C: cat .pes/refs/heads/main and cat .pes/HEAD

Screenshot 4C
cat .pes/refs/heads/main and cat .pes/HEAD

```
$ cat .pes/refs/heads/main
64b1346763a3bf82cab91172264eb7f8c99b072d0c0f01b9f04f105c9741fc95
$ cat .pes/HEAD
ref: refs/heads/main
```

Final Integration: Full integration test output

Final Integration
Full integration test output

```
$ make test-integration
(Windows verification used the provided script with LF line endings and the compiled pes
binary.)
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 1739ef9ce95c... Initial commit

Log after first commit:
commit 1739ef9ce95c276e15ea9bf5d531b8a34f036a1dcb37ab01b3354889280de8a5
Author: Sai Harshith <PES1UG24CS398>
Date:   1776691384

    Initial commit


--- Second Commit ---
Committed: 892114b25f62... Update file.txt

--- Third Commit ---
Committed: 3d3eef999192... Add farewell

--- Full History ---
commit 3d3eef999192cf628ac9bdcac75ce4c64a9e93ab038c7cfab3613f30604e8f45
Author: Sai Harshith <PES1UG24CS398>
Date:   1776691384

    Add farewell

commit 892114b25f62f7d0d64e3234e24b05399d32097e1d1ba7aa63346bfddbf39286
Author: Sai Harshith <PES1UG24CS398>
Date:   1776691384

    Update file.txt

commit 1739ef9ce95c276e15ea9bf5d531b8a34f036a1dcb37ab01b3354889280de8a5
Author: Sai Harshith <PES1UG24CS398>
Date:   1776691384

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
3d3eef999192cf628ac9bdcac75ce4c64a9e93ab038c7cfab3613f30604e8f45

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/17/39ef9ce95c276e15ea9bf5d531b8a34f036a1dcb37ab01b3354889280de8a5
.pes/objects/3d/3eef999192cf628ac9bdcac75ce4c64a9e93ab038c7cfab3613f30604e8f45
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d669579753 3f50d
.pes/objects/89/2114b25f62f7d0d64e3234e24b05399d32097e1d1ba7aa63346bfddbf39286
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Q5: Branching and Checkout

Q5.1

To implement pes checkout , PES would first verify that .pes/refs/heads/ exists, read the commit hash stored there, update .pes/HEAD to contain ref: refs/heads/, then replace the working directory contents with the tree pointed to by that commit. That means reading the commit object, walking its root tree and subtrees, materializing blob contents as files, creating directories, setting modes, and updating the index to match the checked-out snapshot. The complexity is that checkout is not just changing one ref file: it must safely transform live user files without deleting uncommitted work, must handle files that become directories or directories that become files, and must leave the index, HEAD, and working directory consistent if an error happens halfway.

Q5.2

A dirty checkout conflict can be detected by comparing the working directory against the index before replacing files. For each tracked index entry, stat the path and compare size and mtime to the index metadata; for higher confidence, re-hash the working file as a blob and compare the blob hash to the index hash. Then read the target branch commit tree and check whether that path would be different in the target snapshot. If the working file differs from the index and the target branch also wants to write a different blob at that path, checkout must refuse because it would overwrite uncommitted work. Deleted tracked files and type changes should be treated the same way.

Q5.3

Detached HEAD means .pes/HEAD stores a commit hash directly instead of a symbolic ref such as ref: refs/heads/main. New commits made in this state can still be valid: each new commit points to the detached commit as its parent, and HEAD moves to the new hash. The risk is that no branch name points to those commits, so switching away can make them unreachable by normal branch traversal. A user could recover them by creating a branch at the detached commit hash before leaving, by using the printed commit hash from pes commit or pes log, or, in a fuller system, by consulting a reflog and writing that hash into .pes/refs/heads/recovered.

Q6: Garbage Collection and Space Reclamation

Q6.1

A mark-and-sweep collector would first enumerate all branch refs in .pes/refs/heads and push their commit hashes onto a stack or queue. It would maintain a hash set of reachable object IDs. For each commit popped, mark it, parse it, enqueue its parent commit if present, and enqueue its root tree. For each tree, mark it, parse every entry, enqueue child trees, and mark blob hashes. After marking, walk .pes/objects and delete every object hash not present in the reachable set. A hash set gives O(1) average membership checks. With 100,000 commits and 50 branches, the commit visit count is at most 100,000 if histories share ancestry and the set avoids revisits. The total object visits are commits plus all distinct reachable trees and blobs; in a snapshot-based repository this may be many more than 100,000, but each unique object is still visited once.

Q6.2

Concurrent garbage collection is dangerous because commit creation publishes objects in several steps. A process might write new blob and tree objects, but not yet write or update the final commit/ref. A concurrent GC that starts from current branch refs cannot see those new objects, classifies them as unreachable, and deletes one just before the commit object starts referencing it. The result is a commit whose tree or blob is missing. Real Git reduces this risk with conservative pruning rules, lock files, temp files, grace periods for recent unreachable objects, pack/ref locking coordination, and by avoiding deletion of objects that may have been created recently by another process.

Verification Notes

Local verification was performed on Windows with MinGW. The assignment target is Ubuntu 22.04; for local Windows-only execution I used the OpenSSL include/lib paths, a small win_compat.h include, and a larger Windows linker stack because the provided CLI allocates a large Index on the stack. The source remains compatible with the POSIX workflow described in the guide.